

1. Research Novelty

현재 검사를 선택할 때 장기적인 진단 가치를 반영하기 위해, Why Reasoning Fails to Plan에서 제시한 planning 관점을 가져온다. 이 논문은 LLM이 눈앞의 step-wise reasoning은 잘 수행할 수 있지만, 장기적 planning에서는 local greedy decision에 머무르기 쉽다고 보고, 이를 완화하기 위한 요건으로 explicit lookahead, backward value propagation, limited commitment를 제시한다.

다만 해당 논문은 QA dataset에서 MCTS를 사용한 inference-time planning을 수행하며, planner 자체를 학습시키지는 않는다. 우리의 아이디어는 이 세 요건을 sequential clinical diagnosis로 옮기고, MCTS lookahead로 생성한 patient-specific planning signal을 이용해 LLM planner를 직접 학습시키는 것이다. 즉, 다음 검사를 단순히 그럴듯하게 생성하는 것이 아니라, n-step 이후 진단 상태의 value를 현재 action 학습 신호로 되돌려 장기적 진단 가치를 고려하는 test-selection policy를 만든다.

- | | |
|---|---|
| 1 | Confidence를 직접적인 learning signal로 사용한다. Diagnosis Agent의 calibrated confidence를 state value로 두고, planner가 이 value를 높이는 검사를 선택하도록 학습한다. |
| 2 | 현재 선택한 검사의 장기적 가치를 반영한다. 이번 검사 하나가 바로 결과를 바꾸는지만 보는 것이 아니라, n-step 이후 진단 정확도와 confidence를 얼마나 개선하는지까지 반영한다. |

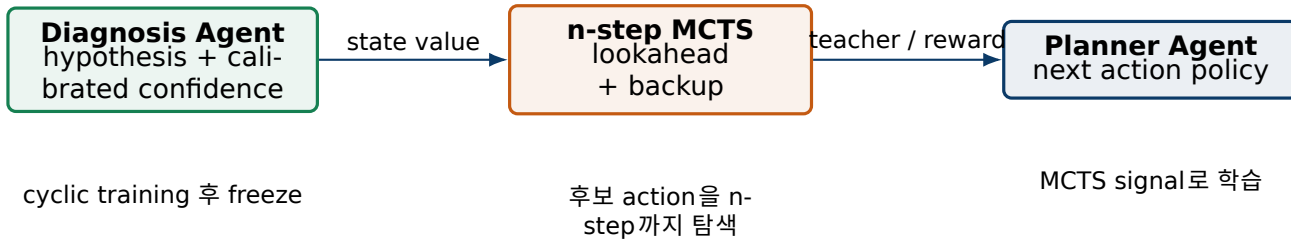
LDTL과의 차이. LDTL은 KL objective로 planner를 posterior에 align한다고 설명하지만, 그 supervision이 logit 기반인지 confidence 기반인지 명확하지 않다. 또한 diagnostic output이 Confident: YES/NO 형태인 것으로 보아, calibrated numerical confidence를 직접 learning signal로 쓰는 구조는 아닌 것으로 보인다.

이를 planning 구조로 만들기 위해 다음 세 요소를 의료 의사결정 과정에 맞게 변환한다.

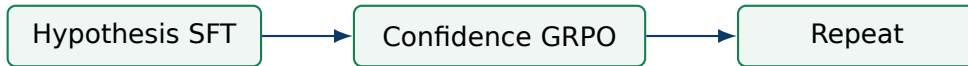
- **Explicit lookahead:** 후보 검사를 n-step까지 가상 진행하고 이후 진단 상태를 평가한다.
- **Backward value propagation:** 미래 진단 value를 현재 action의 가치로 되돌린다.
- **Limited commitment:** 전체 sequence가 아니라 첫 action만 실행하고 다음 state에서 다시 planning한다.

이 세 요소를 넣기 위해 Diagnosis Agent를 먼저 confidence-calibrated evaluator로 만들고, MCTS를 사용해 Planner Agent의 학습 신호를 생성한다.

2. Overall Structure



Diagnosis Agent는 disease hypothesis와 numerical confidence라는 두 목표를 동시에 갖는다. 두 loss를 한 번에 섞기보다 cyclic training으로 번갈아 최적화해, diagnosis prediction과 confidence calibration의 균형을 유지한다.

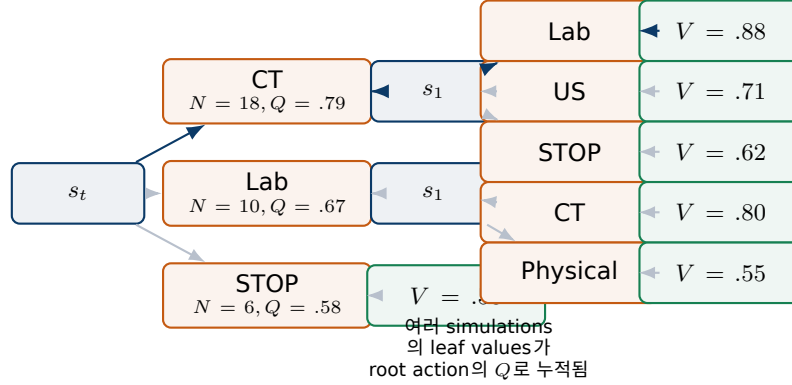


partial state는 full evidence에서 test result를 drop해서 만든다. drop rate를 10%에서 90%까지 점진적으로 키우는 curriculum을 사용해, dense evidence부터 sparse evidence까지 confidence calibration이 유지되도록 한다.

$$s^r = \text{Drop}(s_{\text{full}}, r), \quad r : 10\% \rightarrow 90\%$$

3. n-step MCTS Lookahead

현재 state s_t 에서 action을 즉시 확정하지 않고, 가능한 검사 action들을 tree로 펼친다. 각 branch는 n-step까지 simulate 하고, leaf state의 diagnostic value를 Diagnosis Agent로 평가한다. STOP도 action space에 포함한다. STOP이 선택되면 추가 검사를 simulate하지 않고 현재 state를 leaf로 평가한다.



n-step 뒤 value를 root action으로 되돌린다. 위 예시에서는 여러 simulation을 누적한 결과 CT의 root value가 가장 높으므로 현재 선택은 CT가 된다.

중요한 점은 CT-CBC 전체 sequence를 모두 실행하는 것이 아니라, CT 하나만 실행한다는 것이다. 실제 CT 결과가 state에 추가되면 다시 n-step MCTS를 수행한다.

4. MCTS Signal

MCTS에서 selection formula는 학습 loss가 아니라 search rule이다. 즉 exploration과 exploitation을 조절해 어떤 branch를 더 많이 simulate할지 정한다. 학습에 사용되는 것은 search가 끝난 뒤 만들어지는 root action value와 best action이다.

MCTS 내부 Planner 학습

exploration/exploitation으로 simulation budget을 배분한다.
MCTS가 만든 $Q(s_t, a)$ 또는 a_t^* 를 learning signal로 사용한다.

각 simulation은 현재 state에서 시작해 H -step까지 가상으로 검사 sequence를 진행한다. Leaf state는 freeze된 Diagnosis Agent가 평가한다.

$$D_\theta(s_{\text{leaf}}) \rightarrow (h_{\text{leaf}}, c_{\text{leaf}})$$

$$R_{\text{leaf}} = \begin{cases} c_{\text{leaf}}, & h_{\text{leaf}} = y, \\ -c_{\text{leaf}}, & h_{\text{leaf}} \neq y, \\ 0, & \text{invalid.} \end{cases}$$

현재 state에서 leaf까지 d step이 걸렸다면 step discount를 적용한다.

$$G_{\text{leaf}} = \gamma^d R_{\text{leaf}}$$

따라서 MCTS reward는 단순히 confidence가 큰 상태가 아니라, **정답 진단에 대한 calibrated confidence가 큰 상태**를 선호한다. 또한 $\gamma < 1$ 을 통해 늦게 얻는 confidence를 낮게 평가하므로, 불필요하게 긴 검사 sequence의 value가 감소한다. MCTS는 여러 simulation을 통해 root action별 value를 만든다.

$$Q(s_t, a) = \text{average future value of simulations passing through } (s_t, a)$$

이 값은 leaf reward를 경로에 포함된 action들로 backward propagation한 결과다. 예를 들어 $(s_t \rightarrow \text{image} \rightarrow s_1 \rightarrow \text{lab} \rightarrow s_2)$ 의 leaf reward가 0.9이면, (s_t, image) 와 (s_1, lab) 모두의 value update에 반영된다.

$$a_t^* = \arg \max_a Q(s_t, a)$$

따라서 planner가 받는 signal은 “현재 상태에서 그럴듯한 action”이 아니라, “n-step 뒤 diagnostic value를 가장 크게 개선하는 action”이다.

Hyperparameters. H 는 lookahead depth로, 미래 몇 step까지 simulate할지를 뜻한다. S 는 한 state에서 몇 번 simulation을 돌릴지 정하는 simulation budget이다. γ 는 step discount로, 더 늦게 얻는 diagnostic value를 얼마나 감가할지 정한다. 초기 실험에서는 $H = 2$ 또는 3, $S = 16$ 또는 32, $\gamma = 0.9$ 또는 0.95처럼 시작하고, validation performance와 계산 비용을 기준으로 조정한다. 현재 action space가 작다면 top- k pruning 없이 가능한 action 전체를 탐색 대상으로 둔다.

5. Planner Learning Structure

Planner 학습은 MCTS가 만든 best next action을 text target으로 사용하는 imitation learning으로 구성한다. 즉 planner 자체를 GRPO로 직접 fine-tuning하지 않고, MCTS search를 통해 생성한 patient-specific pseudo label을 distill한다.

MCTS teacher action imitation

MCTS가 고른 root action을 teacher action으로 사용해 Planner를 학습한다. 여기서 supervised learning은 별도의 정답 검사 순서가 있다는 뜻이 아니라, MCTS가 현재 state에서 찾은 best next action을 label처럼 사용한다는 뜻이다. 환자마다, 그리고 같은 환자 안에서도 decision state마다 MCTS를 다시 돌리므로, 하나의 trajectory에서 여러 개의 학습 sample이 나온다.

$$(s_0, a_0^*), (s_1, a_1^*), \dots, (s_T, a_T^*)$$

LLM planner는 최종적으로 text action을 생성한다. 따라서 action classifier를 따로 붙이는 대신, 후보 action string의 conditional log-likelihood를 action prior로 사용할 수 있다.

$$\log p_\phi(["lab"] | s_t), \quad \log p_\phi(["image"] | s_t), \quad \log p_\phi(["stop"] | s_t)$$

$$\mathcal{L}_{\text{SFT}} = -\log \pi_\phi(a_t^* | s_t, h_t, c_t, \mathcal{A}_t)$$

이 단계는 MCTS planning behavior를 Planner Agent에 distill한다. Inference 때는 MCTS를 매번 수행하지 않고도 Planner가 action을 선택할 수 있다.